САМОСТОЯТЕЛЬНАЯ РАБОТА СТУДЕНТА

Дисциплина: Проектирование и применение NoSQL-технологий

Вариант 4. Заказы интернет-магазина

Технологии: MongoDB, Python, Visual Studio Code

Выполнил: Анваров Акрам

Группа: ИС 24-22

Алматы — 2026

1. Цель проекта и предметная область

Разработать работающий учебный прототип учёта заказов интернет-магазина в MongoDB. Каждый заказ содержит встроенный массив позиций (embedding), фиксированную цену на момент покупки (snapshot) и сумму. Реализовать CRUD, восемь запросов, агрегацию выручки по категориям и автоматические тесты.

Сущности

Клиент (customer_id), заказ (_id, created_at, status, total_kzt), позиции заказа (product_id, наименование, категория, зафиксированная цена, количество и стоимость строки). Заказ выступает агрегатом, позиции включены непосредственно в документ.

Бизнес-правило

Для расчётов выручки учитываются только оплаченные, отправленные и доставленные заказы: paid, shipped, delivered. Незавершённые и отменённые заказы в выручку не входят. Это правило учебной модели.

Стек

Python 3, PyMongo 4, MongoDB Server, MongoDB Compass для проверки содержимого коллекции, Visual Studio Code для разработки и запуска.

2. Функциональные и нефункциональные требования

Функциональные требования

F1. Создать заказ и встроить в него одну или несколько товарных позиций.

F2. Сохранить snapshot цены, названия и категории каждой позиции.

F3. Получить заказ по идентификатору.

F4. Найти заказы клиента с сортировкой по дате.

F5. Найти заказы по статусу и временному интервалу.

F6. Обновить статус заказа.

F7. Удалить тестовый заказ.

F8. Рассчитать выручку по категориям и продажи товаров.

Нефункциональные требования

N1. Воспроизводимость: один скрипт создаёт 36 синтетических заказов.

N2. Корректность: валидировать количество товара и неотрицательность цены; автоматические тесты.

N3. Безопасность: не хранить реальные персональные данные и секреты в репозитории.

N4. Поддерживаемость: разделение доменной логики, запросов, доступа к БД и CLI.

3. Карта Access Patterns (8 запросов)

Q1 — Заказы клиента. Вход: customer_id. Ожидаемый результат: Список новых к старым. Приоритет: высокий.

Q2 — Детали заказа. Вход: _id. Ожидаемый результат: Один документ. Приоритет: высокий.

Q3 — Заказы по статусу. Вход: status. Ожидаемый результат: Список по дате. Приоритет: средний.

Q4 — Заказы за период. Вход: created_at range. Ожидаемый результат: Список по дате. Приоритет: средний.

Q5 — Расходы клиента. Вход: customer_id, status. Ожидаемый результат: Сумма оплаченных. Приоритет: средний.

Q6 — Выручка категорий. Вход: status, items[]. Ожидаемый результат: Суммы по категориям. Приоритет: высокий.

Q7 — Топ товаров. Вход: status, items[]. Ожидаемый результат: Топ по количеству. Приоритет: средний.

Q8 — Выручка по дням. Вход: status, created_at. Ожидаемый результат: Сумма по дням. Приоритет: средний.

Два нетривиальных запроса

Q6 выполняет фильтрацию по состоянию заказа, разворачивание items через $unwind, группировку по category_snapshot через $group и сортировку по выручке. Q7 аналогично агрегирует количество проданных единиц товара, суммарную стоимость и формирует рейтинг.

4. Выбор базы данных и архитектура

Выбрана документная NoSQL СУБД MongoDB. Один заказ и его позиции удобно извлекать единым документом без JOIN. Для аналитики применяется aggregation pipeline. Python CLI обращается к модулю запросов и репозиторию, а затем к MongoDB.

Альтернативы

PostgreSQL: строгие связи и транзакции, но отдельные таблицы заказов и позиций с JOIN. Redis: эффективный кэш, но не выбран как основное хранилище истории заказов. MongoDB: естественная модель вложенных товарных позиций, чтение заказов одним документом.

ADR

Решение ADR-001: MongoDB с коллекцией orders; исторические параметры позиций копируются в заказ при покупке. Причина — быстрый доступ к карточке заказа и корректная история цен. Компромисс — денормализация; изменив цену в каталоге, нельзя автоматически менять старые заказы. Подробности — docs/ADR.md.

Архитектурная схема

VS Code/CLI → Python (domain, repository, queries) → MongoDB Server (база sro4_shop, коллекция orders). Отдельно MongoDB Compass служит визуальному контролю. Исходная диаграмма приложена как docs/architecture.svg.

5. Документ MongoDB и индексы

```
{
  "_id": "ORD-001", "customer_id": "C001",
  "created_at": ISODate("2026-10-01T08:00:00Z"),
  "status": "paid", "total_kzt": 12990,
  "items": [{"product_id": "PR01",
    "name_snapshot": "Наушники",
    "category_snapshot": "Электроника",
    "unit_price_kzt_snapshot": 12990,
    "quantity": 1, "line_total_kzt": 12990}]
}
```

Это иллюстрация структуры: реальные значения каждого документа вычисляются скриптом scripts/seed.py. Для денежных величин применяются целые тенге, чтобы избежать неточности float.

Индексы

customer_id + created_at DESC — история заказов клиента; status + created_at DESC — выборка по статусу; items.category_snapshot — возможная фильтрация по категории. Для масштабирования диапазонного поиска по датам может потребоваться индекс created_at.

Snapshot цен

В create_order цены и названия копируются в массив items. После изменения исходного словаря товара сохранённое значение unit_price_kzt_snapshot не изменяется; это проверяется тестом.

6. Реализация и порядок запуска

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
python -m scripts.seed
python -m src.main demo
python -m unittest discover -s tests -v
```

Команды выполняются в терминале Visual Studio Code из корня папки проекта при работающем MongoDB Server. Для проверки состояния зайти в MongoDB Compass по mongodb://localhost:27017, открыть базу sro4_shop и коллекцию orders.

Файлы проекта

src/domain.py — формирование заказа и snapshot; src/db.py — соединение и индексы; src/repository.py — CRUD; src/queries.py — восемь запросов; src/main.py — интерфейс CLI; scripts/seed.py — повторяемая загрузка 36 документов; tests/ — проверки.

CRUD-демонстрация

```
python -m src.main create --id DEMO-1000
python -m src.main get --id DEMO-1000
python -m src.main update --id DEMO-1000 --status delivered
python -m src.main delete --id DEMO-1000
```

7. Демонстрационные запросы

Рабочая база: MongoDB sro4_shop, коллекция orders. Тестовые заказы содержат идентификатор клиента, дату, статус, массив встроенных позиций items и итоговую сумму в тенге.

Q1 — Заказы клиента

{ "customer_id": "C001" } — найдено 5 документов.

Q2 — Детали заказа

Просмотр ORD-001 с раскрытым массивом items и snapshot цены 12 990 ₸.

Q3 — Заказы по статусу

{ "status": "paid" } — найдено 12 документов.

Q4 — Заказы за период

created_at от ISODate("2026-10-01T00:00:00Z") до 2026-10-06 (не включая конец): 15 документов.

Q5 — Анализ трат клиента

Комбинированный фильтр по C001 и paid дал 2 заказа по 12 990 ₸; сумма 25 980 ₸.

Q6 — Выручка по категориям

Aggregation pipeline: $match (paid/shipped/delivered) → $unwind(items) → $group → $sort.

Q7 — Заказы с несколькими позициями

{ "items.1": { "$exists": true } } — найдено 24 заказа с двумя и более позициями.

Q8 — Доставленные заказы

{ "status": "delivered" } — поиск выполнен; видны ORD-008, ORD-032, ORD-020, ORD-026, ORD-014.

В Python CLI дополнительно реализованы Q1–Q8, включая ТОП товаров и выручку по дням.

8. Тестирование и операции CRUD

Проверка в VS Code

Команда python -m pytest -v выполнена успешно: 7 passed in 0.12s. Проверены генерация 36 уникальных заказов, фиксация цены snapshot, расчёт сумм, валидация входных данных и тестовые сценарии CRUD.

CRUD в локальной MongoDB

CREATE	python -m src.main create --id DEMO-1000
READ	python -m src.main get --id DEMO-1000
UPDATE	python -m src.main update --id DEMO-1000 --status paid
DELETE	python -m src.main delete --id DEMO-1000

Проверенные данные

В ходе демонстрации создан тестовый заказ DEMO-1000, клиент C001, товар «Наушники», 2 шт. по 12 990 ₸, итог 25 980 ₸. Пользователь подтвердил успешное выполнение чтения, обновления и удаления заказа.

Ограничения тестирования

Семь автоматических тестов — модульные. Скриншоты Compass подтверждают подключение и результаты чтения реальных данных. Отдельная автоматическая интеграционная тестовая среда не заявлена; для промышленной системы потребуются контроль переходов статусов, безопасность, аудит и нагрузочные тесты.

9. Локальная проверка и выводы

Подтверждено на компьютере студента

- MongoDB Compass: подключение localhost:27017, база sro4_shop, коллекция orders.
- Сохранено 36 тестовых документов; четыре индекса со статусом READY.
- Успешно выполнены фильтры по покупателю, статусу, категории, дате и сумме.
- Фильтр по периоду 1–5 октября 2026 года: 15 документов.
- Выполнена агрегация выручки по пяти товарным категориям.
- В VS Code тестовый запуск pytest завершился: 7 passed.
- CRUD-демонстрация заказа DEMO-1000 пройдена по подтверждению студента.

Аналитический результат по категориям

Электроника	**588 530 ₸**
Одежда	**396 480 ₸**
Аксессуары	**167 880 ₸**
Книги	**115 700 ₸**
Канцелярия	**19 000 ₸**
ИТОГО	**1 287 590 ₸**

Вывод

Реализован прототип системы заказов интернет-магазина на Python и MongoDB. Встроенные позиции заказа фиксируют название, категорию и цену в момент покупки, что сохраняет историю продаж. Аналитика построена на aggregation pipeline. Демонстрация, запросы и тесты выполнены на реальном локальном стенде.

Приложение: скриншоты реального запуска

Рис. 1.1 — Коллекция orders: 36 документов

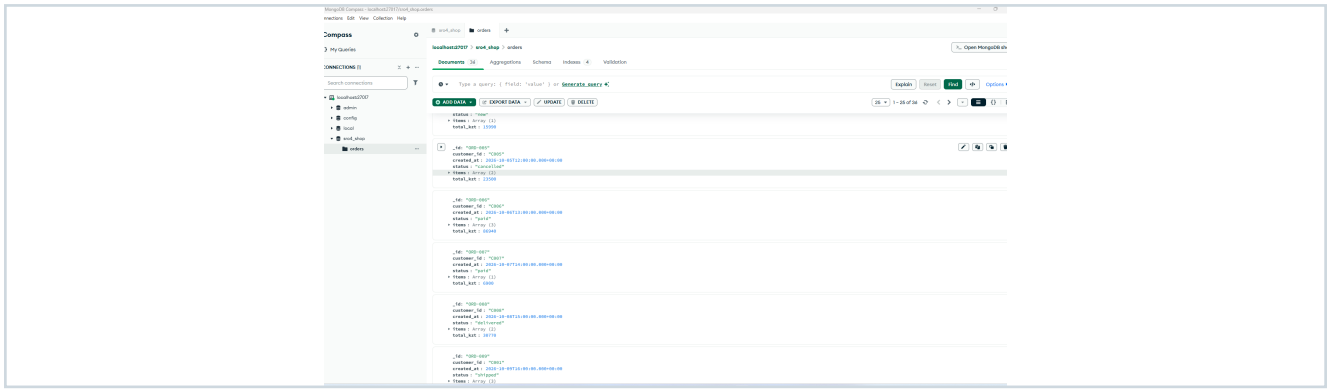


Рис. 1.2 — Индексы: 4 READY

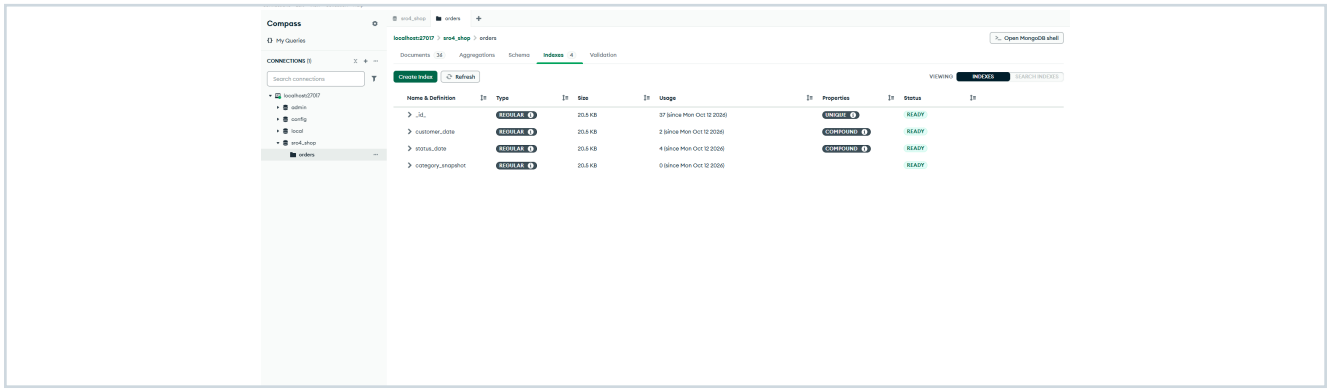


Рис. 1.3 — Тестирование в VS Code: 7 passed

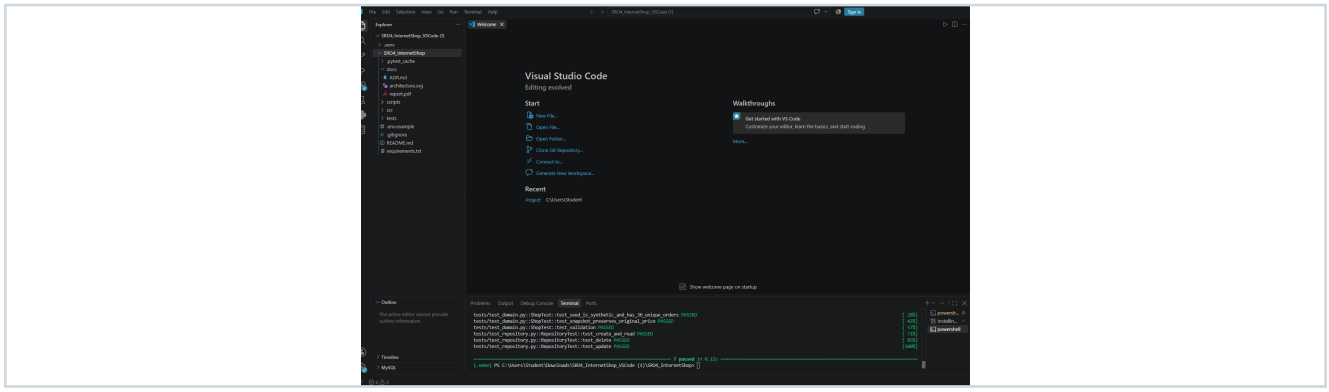
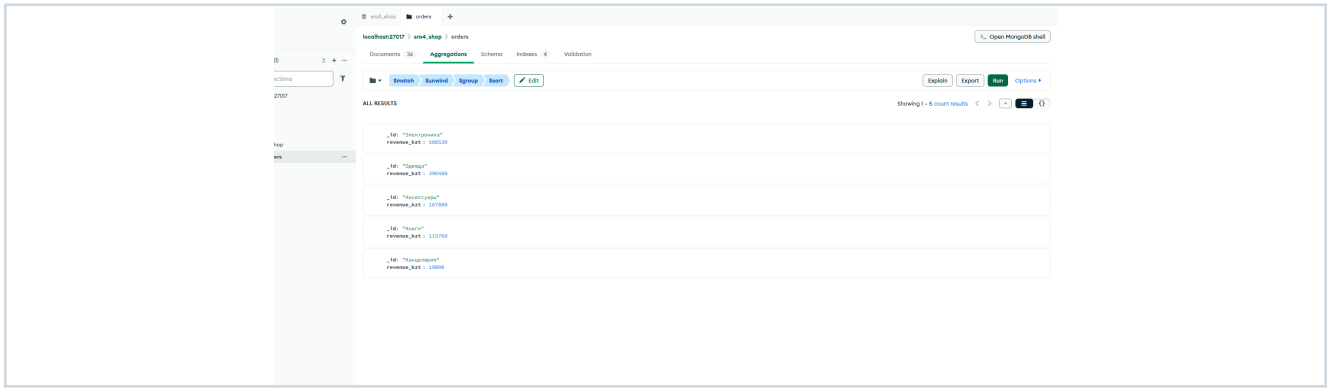


Рис. 1.4 — Выручка по категориям: 1 287 590 ₸



Приложение: скриншоты реального запуска

Рис. 2.1 — Запрос: 5 заказов покупателя C001

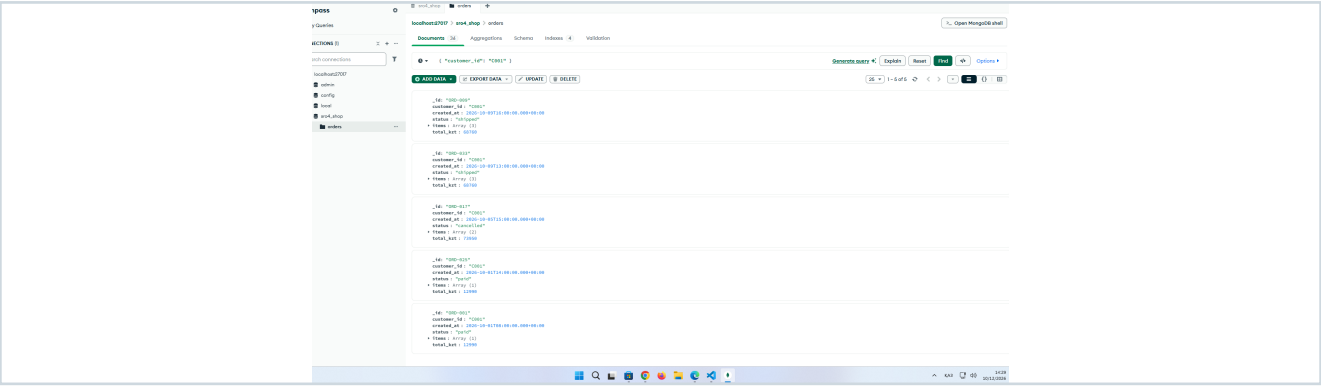


Рис. 2.2 — Фильтр по дате: 15 заказов

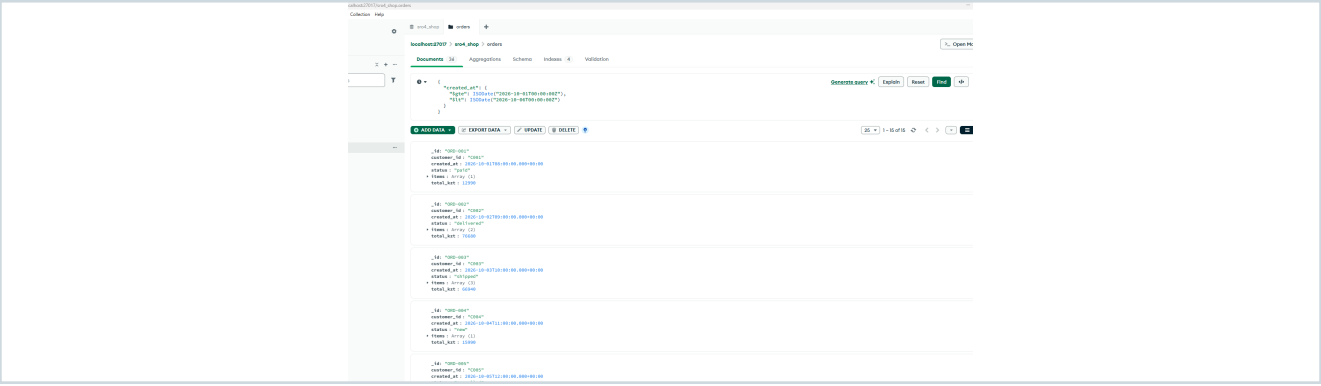


Рис. 2.3 — Заказы с 2+ позициями: 24 документа

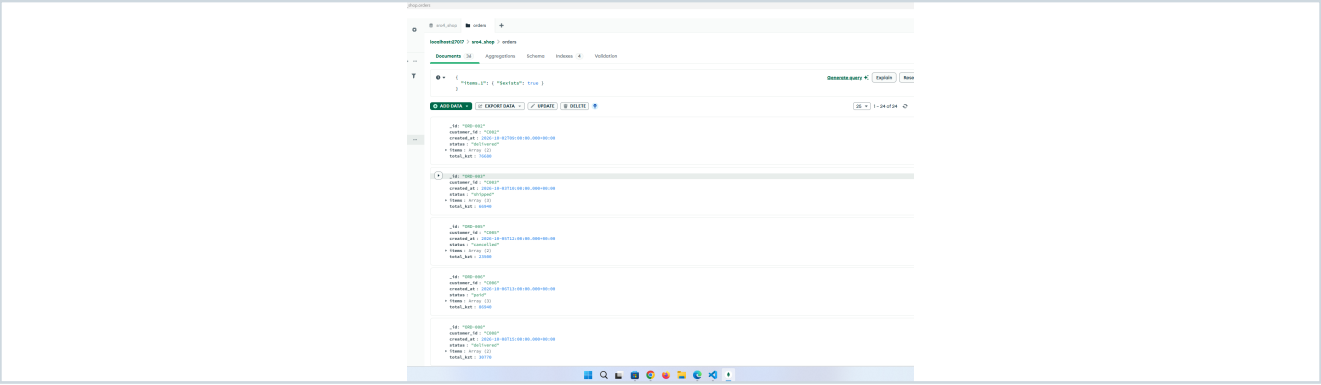


Рис. 2.4 — CREATE: тестовый заказ DEMO-1000

